SIMATS ENGINEERING
ASSESSMENT TOOL 2
Experiments

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1153

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

I Y. Abhay-192411219 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Y. Abhay

Case study

QUESTIONS: 4 **Total Marks:50**

Annexure A

Case study

(Reg. No. 192411219)

a) Online Banking System
A bank wants to develop a secure online banking platform where customers can manage accounts, transfer funds, pay bills, and apply for loans. The system must handle concurrent users, high-volume transactions, and strict regulatory security requirements.

Question: Analyze the system using object-oriented principles. Identify key classes, their attributes, methods, and relationships. Recommend an SDLC model suitable for a security-critical financial application and justify your choice.

b) Smart Library Management System
A university library plans to automate its operations including book cataloguing, member registration, book lending, return tracking, and overdue fine calculation. The system should support both physical and digital book management.

Question: Apply object-oriented concepts to model this system. Identify appropriate classes, relationships (association, aggregation, inheritance), and suggest how polymorphism can be used. Select an SDLC model that suits an institution with defined, stable requirements and justify your answer.

(a) Online Banking System

Object-Oriented Analysis

Key Classes

1. Customer

Attributes:

- customerID
- name
- address
- phoneNumber

Methods:

- login()
- viewAccount()
- transferFunds()
- payBill()
- applyLoan()

2. Account

Attributes:

- accountNumber
- balance
- accountType

Methods:

- deposit()
- withdraw()
- getBalance()

3. Transaction

Attributes:

- transactionID
- amount

- date
- transactionType

Methods:

- processTransaction()
- validateTransaction()

4. Loan

Attributes:

- loanID
- loanAmount
- interestRate
- status

Methods:

- applyLoan()
- approveLoan()
- calculateEMI()

5. BillPayment

Attributes:

- billID
- amount
- dueDate

Methods:

- payBill()
- generateReceipt()

6. SecurityManager

Attributes:

- userCredentials
- securityLevel

Methods:

- authenticateUser()
- authorizeAccess()
- encryptData()

Relationships

Association

- Customer ↔ Account
- Customer ↔ Loan

Aggregation

- Account contains multiple Transactions

Composition

- SecurityManager is an integral part of Banking System

Inheritance

```
Account
  ^
----------------------
|               |
SavingsAccount  CurrentAccount
```

Class Diagram Structure

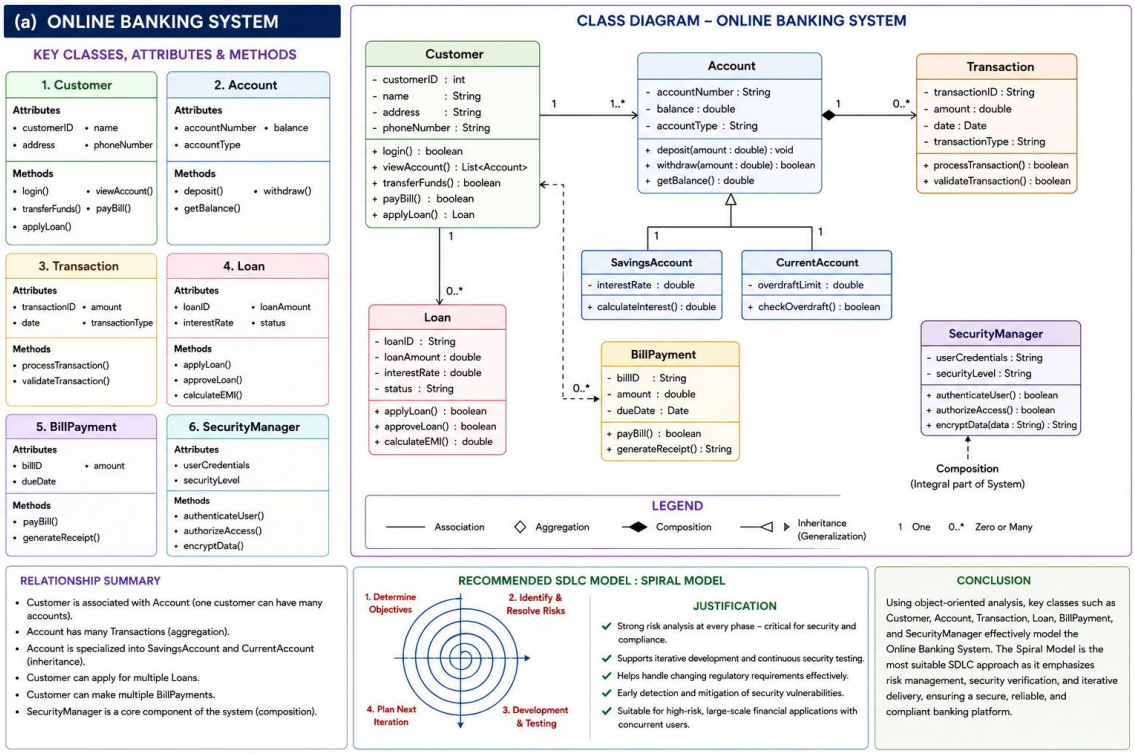
```
Customer ---- Account ---- Transaction
   |
   |
   +---- Loan

Account
  ^
---------------------
|               |
Savings       Current

Banking System ---- SecurityManager
```

Suitable SDLC Model

Spiral Model

Justification

- Strong risk analysis at every phase.
- Suitable for security-critical systems.
- Handles regulatory compliance effectively.
- Supports continuous testing and validation.
- Reduces risk of security vulnerabilities.
- Ideal for large-scale banking applications with frequent updates.



Conclusion

The Online Banking System can be effectively modeled using object-oriented principles through classes such as Customer, Account, Transaction, Loan, and SecurityManager. The Spiral Model is the most suitable SDLC approach because it focuses on risk management, security verification, and iterative development, making it ideal for financial applications.

(b) Smart Library Management System

Object-Oriented Analysis

Key Classes

1. Library

Attributes:

- libraryID
- name
- address

Methods:

- addBook()
- removeBook()
- searchBook()

2. Book

Attributes:

- bookID
- title
- author
- availability

Methods:

- issueBook()
- returnBook()

3. Member

Attributes:

- memberID
- name
- department

Methods:

- borrowBook()
- returnBook()

4. Librarian

Attributes:

- librarianID
- name

Methods:

- manageBooks()
- registerMember()

5. Fine

Attributes:

- fineAmount
- dueDate

Methods:

- calculateFine()
- notifyMember()

Relationships

Association

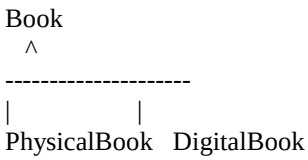
Member -------- Book
Librarian ----- Book

Aggregation

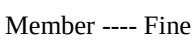
Library <>------ Book

(Books can exist independently of Library records.)

Inheritance

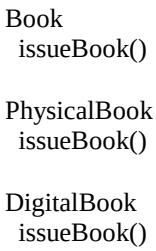


Composition



(Fine is dependent on Member borrowing records.)

Polymorphism Example



The same method **issueBook()** behaves differently:

- Physical Book → Issues a physical copy.
- Digital Book → Provides online access/download.

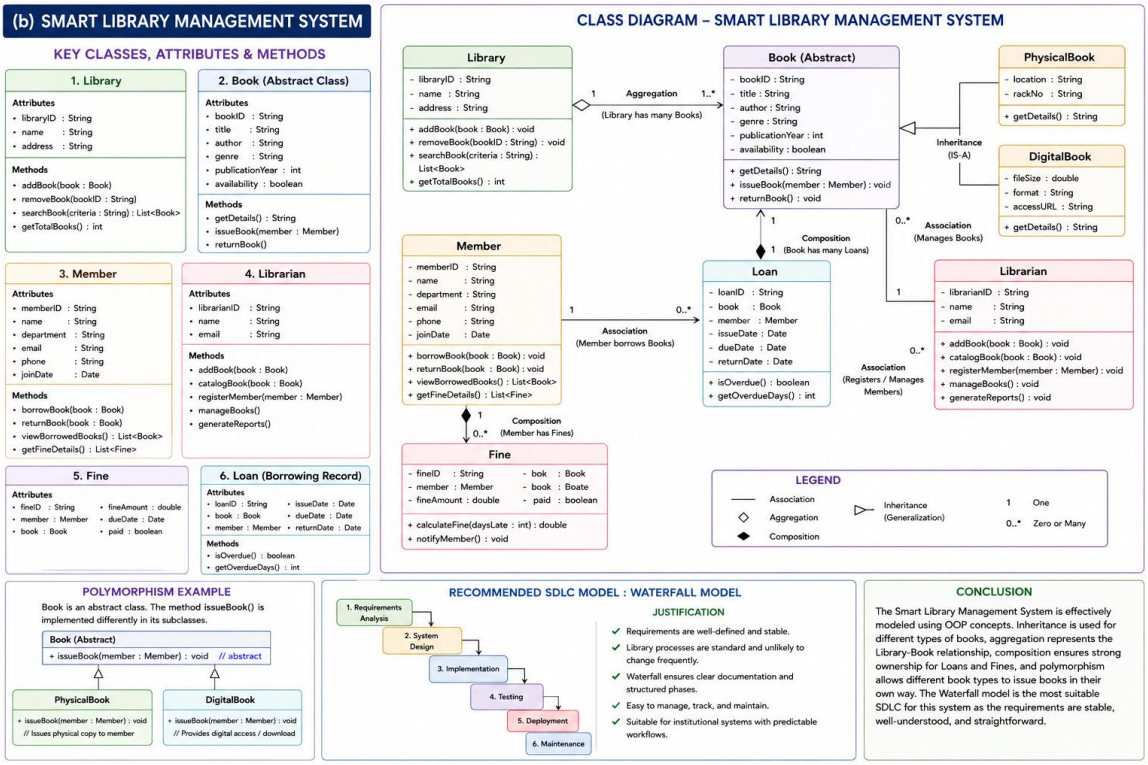
This is **Runtime Polymorphism**.

Suitable SDLC Model

Waterfall Model

Justification

- Requirements are stable and clearly defined.
- University procedures rarely change.
- Easy documentation and maintenance.
- Simple project management.
- Suitable for institutional systems with fixed requirements.



Conclusion

The Smart Library Management System can be modeled using classes such as Library, Book, Member, Librarian, and Fine. Object-oriented concepts like inheritance, aggregation, association, and polymorphism help create a flexible and reusable design. Since the requirements are stable and well-defined, the Waterfall Model is the most suitable SDLC approach for this system.

RUBRICS FOR CALCULATION:

Case Study

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation